

Reviewer Pack — Slice Rank Lower Bounds for Communication Complexity of CNF ...

Entry b4e06614cd46 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 22:03:47 UTC

Slice Rank Lower Bounds for Communication Complexity of CNF Formulas

Entry ID: b4e06614cd46 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 22:03:47 UTC

1. Conjecture

Field A (mathematical branch): Slice rank (algebraic complexity theory) **Field B** (complexity object): Deterministic communication complexity

Statement:

For any CNF formula with m clauses, the deterministic communication complexity of its associated function is at least $\log(\text{slice_rank}(M))$, where M is the incidence matrix of the formula.

Rationale (proposer's reasoning):

Slice rank captures the structural complexity of the incidence matrix, which directly relates to the information-theoretic limits of distributed computation. This bridge may expose hidden algebraic constraints in communication protocols.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e37689cda7ba10ca

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_3cnf(n, m):
    clauses = []
    for _ in range(m):
        clause = [random.choice([-i, i]) for i in range(1, n + 1)]
        random.shuffle(clause)
        clauses.append(clause)
    return clauses

def incidence_matrix(clauses, n):
    m = len(clauses)
    M = [[0] * m for _ in range(2 ** n)]
    for i in range(2 ** n):
        assignment = [(i >> j) & 1] * 2 - 1 for j in range(n)]
        for j, clause in enumerate(clauses):
            if any(assignment[abs(lit) - 1] * lit > 0 for lit in clause):
                M[i][j] = 1
    return M

def slice_rank(M):
    n, m = len(M), len(M[0])
    rank = 0
    for i in range(n):
        if any(M[j][i] == 1 for j in range(m)):
            rank += 1
            for j in range(m):
                if M[i][j] == 1:
                    for k in range(n):
                        M[k][j] = 0
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""
```

```

for n in n_values:
    m = random.randint(2 * n, 4 * n)
    clauses = generate_3cnf(n, m)
    M = incidence_matrix(clauses, n)
    sr = slice_rank(M)
    dcc = math.log(sr)

    total_metric_value += dcc
    instances_tested += 1

mean_dcc = total_metric_value / instances_tested
support_fraction = instances_tested / len(n_values)

if support_fraction < 0.8:
    conjecture_holds = False
    counterexample = "support_fraction_below_80"

return {
    "metric_name": "deterministic_communication_complexity",
    "metric_value": mean_dcc,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]]
    if not seeds:
        from sympy.ntheory import primerange
        seeds = list(primerange(2, 30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\\"seed\\": {seed}, **{result}}}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value / len(results):.2f} std=0.00 support_fr")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={total_metric_value / len(results):.2f} std=0.00 support_fr")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\\"support_fraction_below_80\\" first_failing_seed=")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1558b67d.py", line 90, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1558b67d.py", line 60, in run_trial
    sr = slice_rank(M)
        ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1558b67d.py", line 40, in slice_rank
    if any(M[j][i] == 1 for j in range(m)):
        ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1558b67d.py", line 40, in <genexpr>
    if any(M[j][i] == 1 for j in range(m)):
        ~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError, preventing evaluation of support fraction. The pre-registered support condition cannot be verified. | next: Fix matrix indexing in slice_rank function to handle edge cases properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	35823
2	preregistration	ollama_remote	qwen3:8b	0	0	24136
3	novelty	ollama_remote	qwen3:8b	0	0	20676
4	novelty	ollama_remote	qwen3:8b	0	0	17965
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15775
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9539
7	judge	ollama_remote	qwen3:8b	0	0	17115

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 141030 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in [research/audit/b4e06614cd46.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b4e06614cd46.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b4e06614cd46.tar.gz` (if generated)